

AUX ARMES

Modélisation et réalisation d'un « Universal Wargame » en C++

Soutenance de Projet de Fin d'Année (Licence 3 Informatique)

Présenté par : Naïm COURBOIS & Simon BEASSE

Sous la direction de : M. Igor STEPHAN • Université d'Angers

QU'EST-CE QU'UN WARGAME ?

SIMULATION TACTIQUE ET STRATÉGIQUE

- Simulation d'affrontements militaires ou de conflits en tour par tour.
- Terrain traditionnellement modélisé par une **grille hexagonale** pour des mouvements réalistes à 360°.
- Unités militaires dotées de caractéristiques uniques (mouvement, portée, puissance de feu, logistique).

COMPLEXITÉ ET FACTEURS RÉALISTES

- Prise en compte de variables avancées : le relief topologique, le ravitaillement en munitions/carburant.
- Modélisation de l'état psychologique des troupes (morale, fatigue, panique).
- Objectif général : Développer la réflexion stratégique et l'aide à la décision.

INTRODUCTION ET OBJECTIFS

LE CONTEXTE DU PROJET

- Stage de fin d'année de Licence 3 Informatique (durée de 8 semaines).
- Projet encadré par M. Igor STEPHAN à l'Université d'Angers.
- Ambition : Créer un moteur de wargame universel écrit en C++.

LES DEUX OBJECTIFS MAJEURS

- **Primitives universelles** : Définir des abstractions applicables à n'importe quelle époque ou genre (Espace, Pirates, Historique).
- **Module tactique** : Concevoir un système robuste combinant la gestion de type *Civilization* et les affrontements tactiques inspirés de *Total War*.

ORGANISATION DU PROJET

MÉTHODOLOGIE DE TRAVAIL

- **Phase d'analyse (Semaine 1) :** Conceptualisation de l'architecture logicielle, spécification des mécaniques de base et croquis géométriques.
- **Gestion de version :** Utilisation rigoureuse de Git et GitHub avec branches de fonctionnalités dédiées pour éviter les régressions.

PLANIFICATION (DIAGRAMME DE GANTT)

- **Semaines 2-3 :** Initialisation du code, création du plateau hexagonal et architecture de base des villes.
- **Semaines 4-5 :** Modélisation des composants d'unités, système de combat et gestion du moral.
- **Semaines 6-8 :** Intégration de l'interface graphique SFML/ImGui, tests unitaires et recherche en Deep RL.

LE PARADIGME DU DESIGN PILOTÉ PAR LES DONNÉES

L'IMPÉRATIF D'UNIVERSALITÉ

- Refus de coder des données de gameplay en dur (types d'unités, statistiques, coûts).
- Adoption du paradigme **Data-Driven Design**.
- Découplage total entre la logique algorithmique pure et les configurations de l'univers de jeu.

LE PATRON FABRIQUE (FACTORY PATTERN)

- Lecture de fichiers externes au lancement de l'application pour charger les règles.
- Instanciation dynamique des objets en mémoire selon le contexte choisi.
- Permet d'équilibrer le gameplay ou de changer radicalement d'univers (Espace vs Pirates) sans aucune recompilation du code source.

ARCHITECTURE DU PLATEAU HEXAGONAL

MODÉLISATION POLYMORPHE

- **Hexa** : Classe mère abstraite définissant l'interface universelle (franchissement, coût de déplacement, symbole d'affichage).
- **Tuile_Configurable** : Classe fille concrète instanciée dynamiquement par la fabrique à partir du fichier JSON.
- **Optimisation RAM** : Partage des attributs redondants via un pointeur unique vers une structure `tuile_data` pour éviter les duplications inutiles en mémoire.

GÉNÉRATION PROCÉDURALE ET CONTENEUR

- Génération du terrain par bruit de Perlin en intégrant la bibliothèque open-source `FastNoiseLite`.
- Stockage sous forme de matrice sécurisée :
`std::vector<std::vector<std::unique_ptr<Hexa>>>`.
- Accès direct aux cellules en temps constant $O(1)$ via la méthode `getCell(i, j)` pour des performances optimales.

GESTION ET COMPORTEMENT DES UNITÉS

LE DÉFI GÉOMÉTRIQUE DE L'ORIENTATION

- Une grille stockée en liste de listes possède nativement 8 voisins, alors qu'un hexagone en possède strictement 6.
- Création du module `orientation` : gestion des décalages de coordonnées selon la parité de la ligne (paire vs impaire).
- Modélisation d'un cône de vision réaliste (`Case_visible` et `ConeVision`) propageant le champ de vision selon une portée.

L'ARCHITECTURE ORIENTÉE COMPOSANTS

- Échec de l'héritage classique face aux unités hybrides (ex: véhicule amphibie ou de soutien).
- La classe `Unit` devient un conteneur générique d'attributs communs (PV, PA, position).
- Injection dynamique des capacités via une liste de composants polymorphes héritant de `IComportement` (Mouvement, Attaque, Défense, Soins, Transport, Furtivité).

EXTENSIBILITÉ PAR PROGRAMMATION GÉNÉRIQUE

DÉCORATEURS TRANSVERSAUX

- Éviter l'explosion combinatoire de sous-classes en utilisant les **templates C++**.
- Le template `AvecCooldown` : Enveloppe de manière transparente n'importe quel comportement pour lui adjoindre un temps de récupération.
- Le template `AvecConsommable` : Associe un coût en ressources (munitions, carburant) à l'exécution d'une action.

LOGIQUE LOGISTIQUE ET FACTORY

- `CompRavitaillement` : Permet la création d'unités logistiques capables de transférer des ressources aux troupes sur le terrain.
- `JsonUniteReader` : Parse le fichier de configuration JSON et applique dynamiquement les décorateurs génériques lors de la détection des clés correspondantes.

LE SYSTÈME DE COMBAT ET MODÉLISATION DU MORAL

RÉSOLUTION TACTIQUE

- Validation rigoureuse des conditions (posture, furtivité de la cible, portée réelle).
- **Attaque surprise** : Si l'attaquant agit hors du cône de vision du défenseur, les dégâts de flanc sont multipliés par 1.5 et l'impact psychologique est amplifié.
- Prise en compte séquentielle des composants défensifs (CompDefArmure, CompDefBouclier) pour obtenir les dégâts nets.

L'ÉTAT PSYCHOLOGIQUE ÉVOLUTIF

- Le moral évolue sur une échelle réactive de -20.0 à +20.0 selon les succès et revers.
- Calcul d'un ratio déterminant le palier de l'unité :
 - **Fuite (< 14%)** : Désertion et destruction de l'unité.
 - **Panique / Peur (< 42%)** : Malus offensifs et pertes de PV.
 - **Héroïsme (> 72%)** : Bonus offensif massif de x1.5.

CHAÎNE DE COMMANDEMENT ET MODÉLISATION ÉCONOMIQUE

AURAS TACTIQUES DES OFFICIERS

- Transtypage dynamique pour relier les troupes régulières (Rank_Regulier) à un officier actif (Rank_Commandant).
- Propagation d'auras passives en temps réel apportant des bonus de dégâts, de défense ou de régénération médicale aux unités adjacentes.

GESTION URBAINE ET ÉCONOMIQUE

- Implémentation conjointe des patrons **Prototype** (utilisation de `clone()`) et **Fabrique** pour instancier les bâtiments sans connaître leur classe exacte à la compilation.
- Sécurisation mémoire absolue via `std::make_unique` et utilisation exclusive de `size_t` pour la manipulation des indices dans la classe `City`.

L'ARBITRE LOGIQUE ET LES CONDITIONS DE VICTOIRE

CENTRALISATION DES RÈGLES DE GESTION

- Découplage total : la classe Joueur sert uniquement de conteneur d'état, l'Arbitre agit comme juge suprême.
- L'Arbitre valide en amont chaque intention (solvabilité financière, légalité territoriale, portée géométrique).

PATHFINDING ET VICTOIRE MODULAIRE

- Intégration de l'**algorithme de Dijkstra** pour propager les coûts de déplacement exacts selon la topologie des hexagones.
- Modélisation générique des conditions de victoire via VictorySet et WinConditions, lues depuis le fichier JSON des règles pour valider la fin de partie à chaque tour.

REFACTORISATION MAJEURE : ARCHITECTURE MVC ET PATRON COMMAND

STANDARDISATION JSON

- Abandon des fichiers .txt complexes au profit du format JSON via la bibliothèque `nlohmann/json`.
- Optimisation du parsing par des itérations par référence constante (`const auto&`) éliminant les copies CPU inutiles.

LE COUPLE C++17 : `STD::VARIANT` ET `STD::VISIT`

- Remplacement de l'ancienne structure d'action mal typée.
- Encapsulation de chaque ordre dans des structures légères réunies dans le variant unique `CommandeJeu`.
- La méthode `soumettreCommande` traite les requêtes via un visiteur lambda, supprimant les embranchements conditionnels lourds et garantissant une extension immédiate du gameplay.

L'INTERFACE GRAPHIQUE (SFML + IMGUI) ET OPTIMISATIONS

PIVOT TECHNOLOGIQUE

- Abandon d'Unreal Engine, jugé trop lourd et imposant une réécriture complète de notre structure de code source.
- Adoption de **SFML** (abstraction OpenGL 2D performante) et de **Dear ImGui** pour la génération du HUD utilisateur.
- Calcul trigonométrique précis d'imbrication des hexagones ($posY = 1.5 \times R \times i$).

OPTIMISATIONS SYSTÈMES CRITIQUES

- **View Culling** : Interdiction stricte de dessiner les tuiles situées hors du champ de vision de la caméra.
- **Dirty Flag Cache** : Stockage des données complexes de territoire dans `TerritoireCache`. Le recalcul n'a lieu que si un drapeau d'invalidation est activé, ramenant le coût CPU de $O(L \times C \times J)$ à $O(1)$.

INFRASTRUCTURE RÉSEAU, CONSOLE ET WORLDBUILDING

MULTIJOUEUR SYNCHRONE ET DÉPLOIEMENT

- Réseau géré via `sf::TcpSocket`, `sf::TcpListener` et l'encapsulation de paquets typés à taille fixe 32 bits (`sf::Packet`).
- Compilation croisée MinGW pour Windows et assistant d'installation natif généré via Inno Setup.
- Mise en place d'une interface console MVC complète (`main_console.cc`) pour exécuter la boucle de jeu de manière textuelle.

WORLDBUILDING ET PROMPT ENGINEERING

- Création de l'univers spatial de la galaxie de *Lerne* pour tester la gestion de trois factions asymétriques.
- Pipeline d'assets : Décors via *PixelPlanets* et visuels d'unités générés par l'IA *Gemini* en développant une structure de requêtes stricte pour assurer la cohérence artistique.
- Équilibrage économique réalisé sur mesure avec l'IA *Claude*.

VALIDATION ALGORITHMIQUE PAR TESTS UNITAIRES

NÉCESSITÉ DE RIGUEUR

- Complexité mathématique inhérente au wargame (calculs de vision hexagonale, propagation de Dijkstra, résolution de moral).
- Intégration du framework moderne et léger `doctest.h` au sein de `main_test.cc`.

ISOLATION ET ISOLATION PAR BOUCHONS (MOCKING)

- Développement de la fonction utilitaire `createTestUnit` pour générer à la volée des unités virtuelles modulaires.
- Permet de valider isolément la logique pure des algorithmes sans charger l'environnement graphique, le plateau ou les fichiers de configuration JSON externes.
- Garantie d'une non-régression logicielle lors de chaque phase de refactorisation.

INTELLIGENCE ARTIFICIELLE PAR APPRENTISSAGE PROFOND (DEEP RL)

FRAMEWORK ET BINDINGS

- Recherche sur le Deep Reinforcement Learning via l'algorithme **PPO** (Proximal Policy Optimization) de `stable-baselines3` en Python (PyTorch).
- Utilisation de la bibliothèque Pybind11 (`bindings.cc`) pour compiler le moteur C++ en module Python binaire (`wargame_env.so`) et préserver les performances.
- Implémentation de l'interface standard Gymnasium.

CONTRAINTES MATÉRIELLES ET OPTIMISATIONS

- **CPU vs GPU** : Contre-intuitivement, l'entraînement CPU s'est révélé plus rapide que le GPU en raison du goulot d'étranglement PCIe lié au transfert en temps réel des simulations du moteur.
- **Multiprocessing** : Parallélisation des instances de jeu, augmentant la vitesse de 30 FPS à 120 FPS.
- Intégration d'un **coupe-circuit** pour bloquer les boucles infinies d'actions illégales de l'IA.

CONCLUSION ET PERSPECTIVES

BILAN TECHNIQUE ET HUMAIN

- Conception intégrale d'un moteur de jeu d'universal wargame stable, performant et modulaire en C++ moderne.
- Maîtrise approfondie des patrons de conception majeurs (Factory, Prototype, Command, Décorateur, MVC) et des structures de données avancées.
- Compréhension concrète des défis liés au travail collaboratif et à l'unification d'architectures logicielles.

PERSPECTIVES DE DÉVELOPPEMENT

- Finalisation du module d'entraînement Deep RL sur de très longues sessions pour stabiliser la convergence du réseau de neurones.
- Sécurisation de l'infrastructure réseau pour parer aux injections de données.
- Déploiement de nouvelles compétences génériques (attaques de zone, mécaniques d'alliances ou de trahison).

COMPLÉMENTS REQUIS : AJOUT UML

À COMPLÉTER

Emplacement réservé pour l'insertion du diagramme des classes global du système (Unité, Comportements, Moteur, Arbitre, Joueur)

COMPLÉMENTS REQUIS : TEST ÉCRIT EN LATEX

À COMPLÉTER

Emplacement réservé pour l'intégration de la documentation formelle et des résultats mathématiques des tests compilés sous LaTeX